

Pruebas, preparación para el lanzamiento y publicación de la aplicación móvil híbrida

Cosecha Directa

Samuel Felipe Barbosa Ríos

Facultad de Ingeniería, Fundación Universitaria Compensar

Programa de Ingeniería de Software

Materia: Desarrollo de Aplicaciones Móviles Híbridas

Docente: Julio César Bolaño Pérez

24 de septiembre de 2026

Tabla de contenido

Resumen.....	4
Pruebas, preparación para el lanzamiento y publicación de la aplicación móvil híbrida Cosecha	
Directa.....	5
Objetivos	5
Descripción de la aplicación	6
Estrategia de pruebas	7
Resultados	9
Pruebas unitarias y de integración	9
Pruebas de extremo a extremo en el navegador.....	11
Pruebas de extremo a extremo en el emulador de Android	14
Prueba en dispositivo físico	17
Prueba de la compilación de producción	18
Depuración de errores	19
Preparación para el lanzamiento	20
Android: paquete firmado	21
iOS: proyecto nativo	21
Proceso de publicación en las tiendas	22
Google Play Console.....	22
App Store Connect.....	22

Consideraciones de seguridad.....	23
Actualizaciones y mantenimiento	24
Conclusiones	25
Referencias.....	26

Resumen

Este informe documenta las pruebas realizadas a la aplicación móvil híbrida de la tienda en línea Cosecha Directa S.A.S., construida con Ionic, Angular y Capacitor como producto final del proyecto de aprendizaje basado en proyectos. La aplicación implementa autenticación con token, consumo de un API REST, carrito de compras y almacenamiento local. Las pruebas se organizaron en cuatro niveles: 33 pruebas unitarias y de integración con Vitest, una prueba de extremo a extremo de 13 pasos en el navegador, una prueba de extremo a extremo de 8 pasos en el emulador de Android 16 y una prueba de humo sobre la compilación de producción firmada. Todas las pruebas se superaron. La cobertura de la capa de servicios, donde reside la lógica de negocio, alcanzó el 94,5 % de las sentencias. Durante el proceso se detectaron y corrigieron cuatro defectos, uno de ellos capaz de romper la protección de las rutas. El informe describe además la preparación del paquete de Android (AAB firmado) y del proyecto de iOS, el proceso de publicación en Google Play Console y App Store Connect, las consideraciones de seguridad y la estrategia de mantenimiento. La publicación efectiva en las tiendas queda fuera del alcance por no contar con las membresías de desarrollador, tal como lo prevé el enunciado de la actividad.

Palabras clave: aplicación híbrida, Ionic, Capacitor, pruebas de software, publicación en tiendas de aplicaciones.

Pruebas, preparación para el lanzamiento y publicación de la aplicación móvil híbrida

Cosecha Directa

Una aplicación móvil híbrida comparte una sola base de código escrita con tecnologías web y se ejecuta dentro de un contenedor nativo que le da acceso al dispositivo (Capacitor, s.f.-a). Esa promesa de «escribir una vez» no elimina la necesidad de probar en cada plataforma: el mismo código se ejecuta en motores y sistemas de almacenamiento distintos, y hay defectos que solo aparecen en el dispositivo. La experiencia de la etapa de profundización lo confirmó, cuando un error de rutas de archivo pasó inadvertido en el navegador y solo se manifestó en Android.

Este documento corresponde a la etapa final del proyecto de aprendizaje basado en proyectos (ABP) de la asignatura. El producto es la aplicación de la tienda en línea Cosecha Directa S.A.S., una empresa que vende productos agrícolas de pequeños productores de Chocontá, Villapinzón y Ubaté con entrega a domicilio en Bogotá, cuyos requerimientos se especificaron en la etapa de contextualización (Barbosa Ríos, 2026). El informe presenta la aplicación construida, la estrategia de pruebas, los resultados obtenidos en navegador y en emulador, los defectos encontrados, la preparación de los archivos de producción para Android e iOS y el proceso de publicación en las dos tiendas.

Objetivos

Objetivo general. Verificar el funcionamiento de la aplicación móvil híbrida Cosecha Directa en navegador y en Android, y preparar los archivos que se envían a producción en Google Play y en la App Store.

Objetivos específicos.

1. Comprobar con pruebas automatizadas la autenticación, el consumo del API, el carrito de compras y el almacenamiento local.
2. Ejecutar el flujo completo de compra en el navegador y en el emulador de Android, y verificar la persistencia de los datos al cerrar la aplicación.
3. Registrar y corregir los defectos encontrados durante las pruebas.
4. Generar el paquete firmado de Android (AAB) y el proyecto nativo de iOS, y documentar el proceso de publicación en cada tienda.
5. Identificar las consideraciones de seguridad y de mantenimiento que aplican a la aplicación.

Descripción de la aplicación

La aplicación se generó con la plantilla de pestañas de Ionic en su variante de componentes independientes de Angular (Ionic, s.f.). Tiene cinco pantallas: inicio de sesión, catálogo, ficha de producto, carrito y cuenta del usuario. El código se organiza en capas: los **modelos** definen los datos (producto, pedido, usuario); los **servicios** concentran la lógica de negocio y la comunicación con el API; el **guard** y el **interceptor** HTTP resuelven la autenticación de forma transversal, y las **páginas** solo presentan la información. Esta separación permitió probar la lógica sin depender de la interfaz.

Como servidor se usó DummyJSON, un API REST público que ofrece autenticación con tokens JWT, un catálogo de productos y un servicio de carritos (DummyJSON, s.f.). Su catálogo está en inglés y en dólares, por lo que el servicio de catálogo lo adapta al surtido de la tienda: traduce los nombres, asigna la categoría, el municipio de origen y la unidad de venta, y convierte el precio a pesos colombianos. Los productos que la tienda no vende, como alimento para

mascotas, se descartan. La Tabla 1 relaciona los cuatro componentes que exige la actividad con su implementación y con los requerimientos definidos en la etapa de contextualización.

Tabla 1

Componentes exigidos por la actividad y su implementación

Componente	Implementación	Requerimientos
Autenticación	Inicio de sesión contra POST /auth/login; el token se adjunta a cada petición mediante un interceptor, que lo renueva con POST /auth/refresh ante una respuesta 401. Un guard protege las rutas de la tienda.	RF-02, RNF-07
Servicios API	Catálogo desde GET /products/category/groceries; envío del pedido a POST /carts/add; datos del usuario desde la respuesta del inicio de sesión.	RF-05, RF-08
Carrito de compras	Agregar, cambiar cantidades con tope en las existencias, quitar y calcular el total; aviso de valor estimado para productos de peso variable; confirmación del pedido e historial.	RF-10, RF-11, RF-25
Almacenamiento local	Capacitor Preferences guarda la sesión, el carrito, el historial de pedidos y una copia del catálogo para consultarlo sin conexión.	RF-10, RNF-04

Nota. RF = requerimiento funcional; RNF = requerimiento no funcional, según la numeración del documento de requerimientos (Barbosa Ríos, 2026). Elaboración propia.

El almacenamiento local se implementó con el complemento Preferences de Capacitor, que guarda pares de clave y valor en SharedPreferences en Android, en UserDefaults en iOS y en localStorage en el navegador (Capacitor, s.f.-b). Angular 22 funciona sin zone.js, por lo que el estado de la aplicación se manejó con señales: cada cambio en el carrito reemplaza el arreglo completo en lugar de modificarlo, de modo que la interfaz se actualiza de forma confiable.

Estrategia de pruebas

La estrategia siguió la pirámide de pruebas propuesta por Cohn (2009): muchas pruebas unitarias, rápidas y baratas, en la base; menos pruebas de integración en el medio, y pocas pruebas de extremo a extremo, más lentas pero más cercanas a la experiencia del usuario, en la cima. La guía de pruebas de Angular recomienda el mismo enfoque y ofrece utilidades para

aislar los servicios HTTP, como el controlador de pruebas del cliente HTTP (Angular, s.f.). A esa pirámide se añadió una prueba de humo sobre la compilación de producción, porque el paquete que llega a la tienda no es el mismo que se prueba en desarrollo: está firmado con otra llave y tiene deshabilitada la depuración.

Angular 22 genera los proyectos con Vitest como ejecutor de pruebas, en lugar de la combinación de Jasmine y Karma de versiones anteriores. La sintaxis de las pruebas (describe, it, expect) es la misma de Jasmine, de modo que los conceptos de la guía de Angular aplican sin cambios (Vitest, s.f.). El emulador y las herramientas de compilación de Android provienen de Android Studio (Android Developers, s.f.-a). Las pruebas de extremo a extremo se escribieron en Python con Playwright, que automatiza Microsoft Edge en el navegador y, en el emulador, se conecta al WebView de la aplicación mediante el protocolo de depuración de Chrome (Microsoft, s.f.). La Tabla 2 resume los niveles de prueba y la Tabla 3 el entorno utilizado.

Tabla 2

Niveles de prueba aplicados

Nivel	Herramienta	Qué verifica	Casos
Unitarias y de integración	Vitest 4 con jsdom y el controlador de pruebas HTTP de Angular	Servicios, guard, interceptor y páginas de forma aislada, con el API simulado	33
Extremo a extremo en navegador	Playwright con Microsoft Edge, pantalla de teléfono (412 × 915)	Flujo completo contra el API real, recarga de la página y modo sin conexión	13 pasos
Extremo a extremo en emulador	Playwright conectado al WebView por adb; capturas con adb screencap	Flujo completo en Android, cierre forzado del proceso y datos en SharedPreferences	8 pasos
Humo de producción	adb, apksigner y jarsigner	Firma, instalación, arranque, tiempo de inicio y depuración deshabilitada del APK de release	7 verificaciones

Nota. Elaboración propia.

Tabla 3*Entorno de desarrollo y de pruebas*

Componente	Versión
Ionic Framework (@ionic/angular) / Ionic CLI	9.0.5 / 7.2.1
Angular / TypeScript	22.1.7 / 6.0.3
Capacitor (core, android, ios) / Preferences	8.5.2 / 8.0.1
Node.js / npm	22.23.2 / 10.9.8
Vitest / cobertura	4.0.18 / @vitest/coverage-v8
Playwright para Python / Microsoft Edge	1.61.0 / 153
Android Studio / Gradle / Android Gradle Plugin	compilación 253.30387 / 8.14.3 / 8.13.0
Emulador	Medium Phone, Android 16 (API 36.1), x86_64, 1080 × 2400
SDK de compilación / versión mínima	API 36 / API 24 (Android 7.0)
Sistema operativo del equipo	Windows 11 Pro

Nota. Elaboración propia.

Resultados**Pruebas unitarias y de integración**

Se escribieron 33 pruebas distribuidas en nueve archivos (Tabla 4). Las pruebas de los servicios que consumen el API no hacen peticiones reales: usan el controlador de pruebas HTTP de Angular, que intercepta cada petición, permite comprobar su URL, método y cuerpo, y responde con datos controlados o con errores simulados. Así se probaron casos difíciles de reproducir a mano, como un token vencido o la caída de la conexión en medio de una compra.

Tabla 4*Pruebas unitarias y de integración por archivo*

Archivo de prueba	Comportamientos verificados	Casos
carrito.service.spec.ts	Agregar y acumular, total, tope en las existencias, retiro al llegar a cero, aviso de peso variable, persistencia al reabrir y vaciado	7
auth.service.spec.ts	Inicio de sesión, credenciales inválidas, restauración de la sesión, renovación del token, cierre por token inválido y cierre de sesión	6

Archivo de prueba	Comportamientos verificados	Casos
catalogo.service.spec.ts	Adaptación del producto a pesos, descarte de productos ajenos, carga y copia local, modo sin conexión, búsqueda y filtro	5
pedidos.service.spec.ts	Envío al API y vaciado del carrito, conservación del carrito si el API falla, numeración de pedidos, carrito vacío	4
auth.interceptor.spec.ts	Token en las peticiones del API, ningún token hacia otros dominios, renovación y reintento ante un 401	3
auth.guard.spec.ts	Redirección al login sin sesión, acceso con sesión y redirección del login al catálogo	3
login.page.spec.ts	Validación de campos, mensaje de credenciales incorrectas y mensaje de falta de conexión	3
tabs.page.spec.ts / app.component.spec.ts	Contador del carrito en la pestaña y creación de la aplicación	2

Nota. Elaboración propia a partir de los archivos de prueba del repositorio.

La ejecución final superó las 33 pruebas en 5,7 segundos (Figura 1). La cobertura global de las sentencias fue del 91,8 %, y la de la capa de servicios, del 94,5 %, por encima del 70 % que fija el requerimiento RNF-16 para la lógica de negocio. El reporte de cobertura de Vitest solo mide los archivos que las pruebas cargan; las páginas de catálogo, producto, carrito y cuenta no tienen pruebas unitarias propias y se verificaron con las pruebas de extremo a extremo que se describen a continuación.

Figura 1

Ejecución de las pruebas unitarias y reporte de cobertura

```

PS C:\...\cosecha-directa> npx ng test --watch=false --coverage
RUN v4.0.18 C:/Users/elbub/Projects/cosecha-directa
✓ app src/app/services/carrito.service.spec.ts (7 tests) 85ms
✓ app src/app/core/auth.interceptor.spec.ts (3 tests) 57ms
✓ app src/app/services/catalogo.service.spec.ts (5 tests) 61ms
✓ app src/app/services/pedidos.service.spec.ts (4 tests) 64ms
✓ app src/app/services/auth.service.spec.ts (6 tests) 82ms
✓ app src/app/core/auth.guard.spec.ts (3 tests) 57ms
✓ app src/app/app.component.spec.ts (1 test) 75ms
✓ app src/app/tabs/tabs.page.spec.ts (1 test) 133ms
✓ app src/app/pages/login/login.page.spec.ts (3 tests) 262ms
Test Files 9 passed (9)
Tests 33 passed (33)
Duration 5.70s (transform 5.28s, setup 7.46s, import 9.51s, tests 876ms, environment 11.11s)
% Coverage report from v8
-----
File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files | 91.81 | 85.03 | 82.53 | 94.05 |
app | 100 | 90 | 100 | 100 |
...component.html | 100 | 100 | 100 | 100 |
app.component.ts | 100 | 90 | 100 | 100 | 12
app/core | 88.46 | 82.35 | 75 | 95.83 |
auth.guard.ts | 100 | 75 | 100 | 100 | 17
...nterceptor.ts | 81.25 | 84.61 | 66.66 | 92.85 | 25
app/pages/login | 81.81 | 64.28 | 50 | 83.72 |
login.page.html | 73.8 | 0 | 0 | 75 | 18,29,36-37,42-44
login.page.ts | 95.83 | 90 | 100 | 94.73 | 40
app/services | 94.52 | 93.33 | 86.66 | 96.36 |
...to.service.ts | 92.3 | 100 | 100 | 88.88 | 20
auth.service.ts | 97.05 | 77.77 | 100 | 96.29 | 58
...to.service.ts | 94.73 | 93.75 | 89.47 | 95.83 | 47
...go.service.ts | 89.18 | 93.33 | 63.63 | 96.66 | 113
...os.service.ts | 100 | 100 | 100 | 100 |
app/tabs | 100 | 91.66 | 100 | 100 |
tabs.page.html | 100 | 50 | 100 | 100 | 11
tabs.page.ts | 100 | 100 | 100 | 100 |
app/testing | 100 | 100 | 100 | 100 |
datos.ts | 100 | 100 | 100 | 100 |
environments | 100 | 100 | 100 | 100 |
environment.ts | 100 | 100 | 100 | 100 |
-----

```

Nota. Salida del comando `npx ng test --watch=false --coverage`. Captura propia.

Pruebas de extremo a extremo en el navegador

El script `pruebas/e2e_navegador.py` abre la aplicación en Microsoft Edge con una pantalla de teléfono y recorre el flujo de un cliente contra el API real. Cada paso incluye aserciones: si el resultado no coincide, la prueba se detiene. Además, el script registra los errores de JavaScript de la página y falla si aparece alguno. Los 13 pasos se superaron sin errores (Tabla 5).

Tabla 5*Casos de prueba de extremo a extremo en el navegador*

ID	Caso	Resultado esperado	Estado
N-01	Abrir /tabs/carrito sin sesión	Redirige a /login	Superado
N-02	Ingresar con contraseña incorrecta	Mensaje «Usuario o contraseña incorrectos.»	Superado
N-03	Ingresar con emilys / emilyspass	Respuesta 200 de /auth/login y catálogo con 20 productos	Superado
N-04	Buscar «ubaté»	Solo los 3 productos de Ubaté	Superado
N-05	Filtrar por «Frutas»	5 productos	Superado
N-06	Agregar dos productos desde el catálogo	La pestaña del carrito muestra 2	Superado
N-07	Abrir la ficha de la leche y sumar dos unidades	Contador en 2	Superado
N-08	Revisar el carrito	3 productos y total estimado	Superado
N-09	Recargar la página	Sesión, carrito y total intactos; claves sesion, carrito y catalogo en el almacenamiento	Superado
N-10	Confirmar el pedido	Respuesta 201 de /carts/add y alerta «Pedido recibido»	Superado
N-11	Revisar «Mi cuenta»	Nombre del usuario y un pedido en el historial	Superado
N-12	Bloquear el API y abrir el catálogo	Aviso de sin conexión y 20 productos desde la copia local	Superado
N-13	Cerrar sesión y volver al catálogo	Redirige a /login	Superado

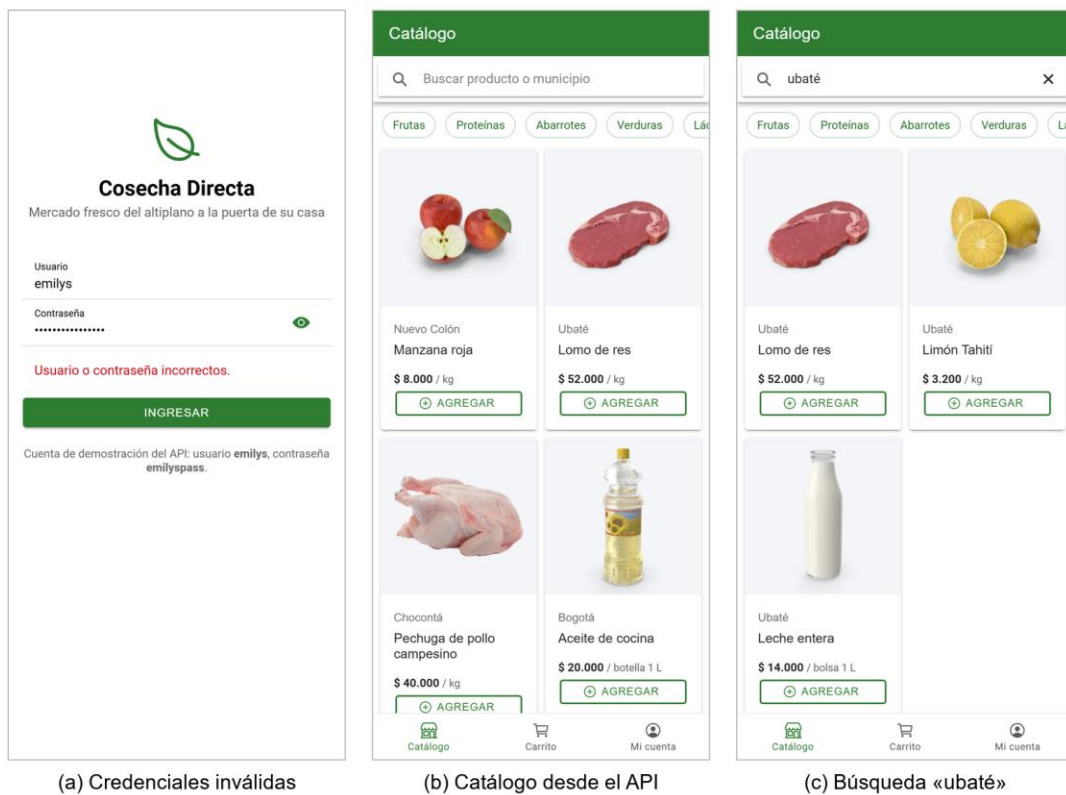
Nota. El caso N-12 simula la falta de red interceptando las peticiones al API desde Playwright.

Elaboración propia.

Las figuras 2 y 3 muestran algunos de estos pasos. El caso N-12 es relevante para el contexto del usuario final: la etapa de contextualización identificó que muchos clientes usan planes de datos limitados y conexiones inestables, y la copia local del catálogo permite seguir consultando los productos cuando la red falla.

Figura 2

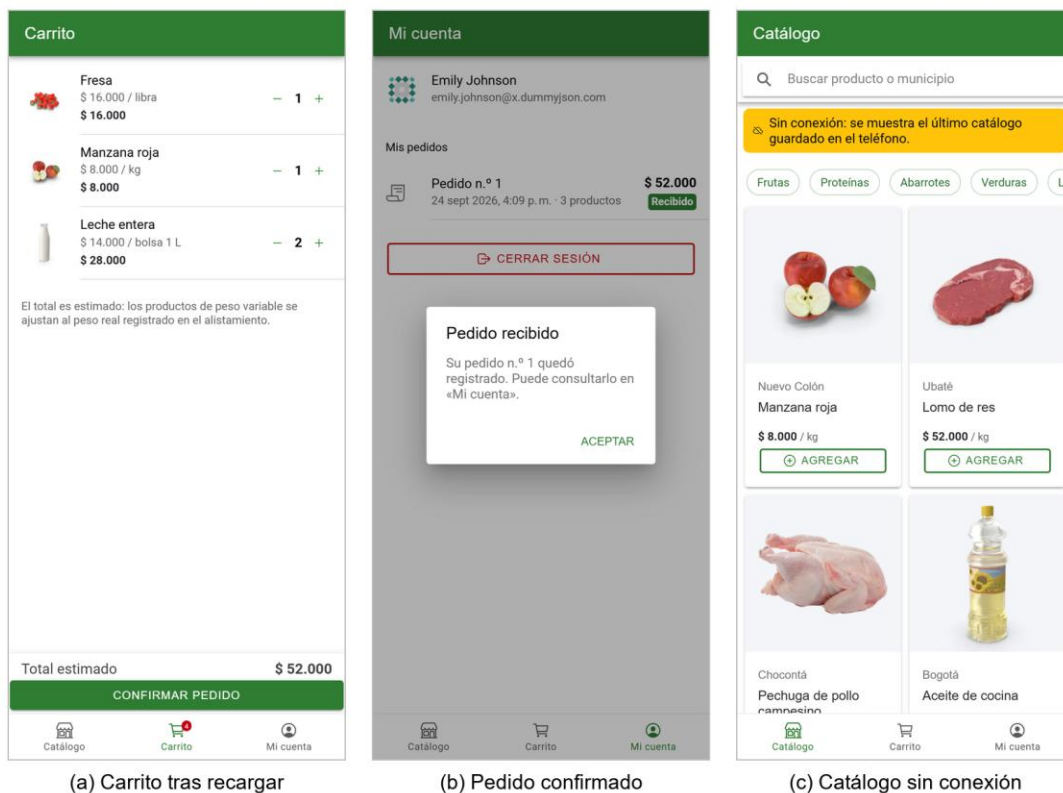
Acceso, catálogo y búsqueda en el navegador



Nota. Capturas tomadas por el script de prueba en Microsoft Edge con una pantalla de 412×915 píxeles. Captura propia.

Figura 3

Persistencia, confirmación del pedido y modo sin conexión en el navegador



Nota. En (a) la página ya se recargó y el carrito se recuperó del almacenamiento local; en (c) el API está bloqueado. Captura propia.

Pruebas de extremo a extremo en el emulador de Android

La compilación de depuración se instaló en el emulador de Android 16. El script pruebas/e2e_emulador.py borra los datos de la aplicación, la abre, se conecta a su WebView a través de adb y ejecuta el flujo de compra. Las capturas se tomaron con adb screencap, por lo que muestran la pantalla real del teléfono. Para comprobar la persistencia en condiciones reales, el script fuerza el cierre del proceso con am force-stop, vuelve a abrir la aplicación y verifica que el carrito siga intacto. Al final lee el archivo de SharedPreferences del dispositivo para confirmar

que la sesión se borró al cerrarla y que el historial de pedidos permanece. Los ocho pasos se superaron (Tabla 6; figuras 4 y 5).

Tabla 6

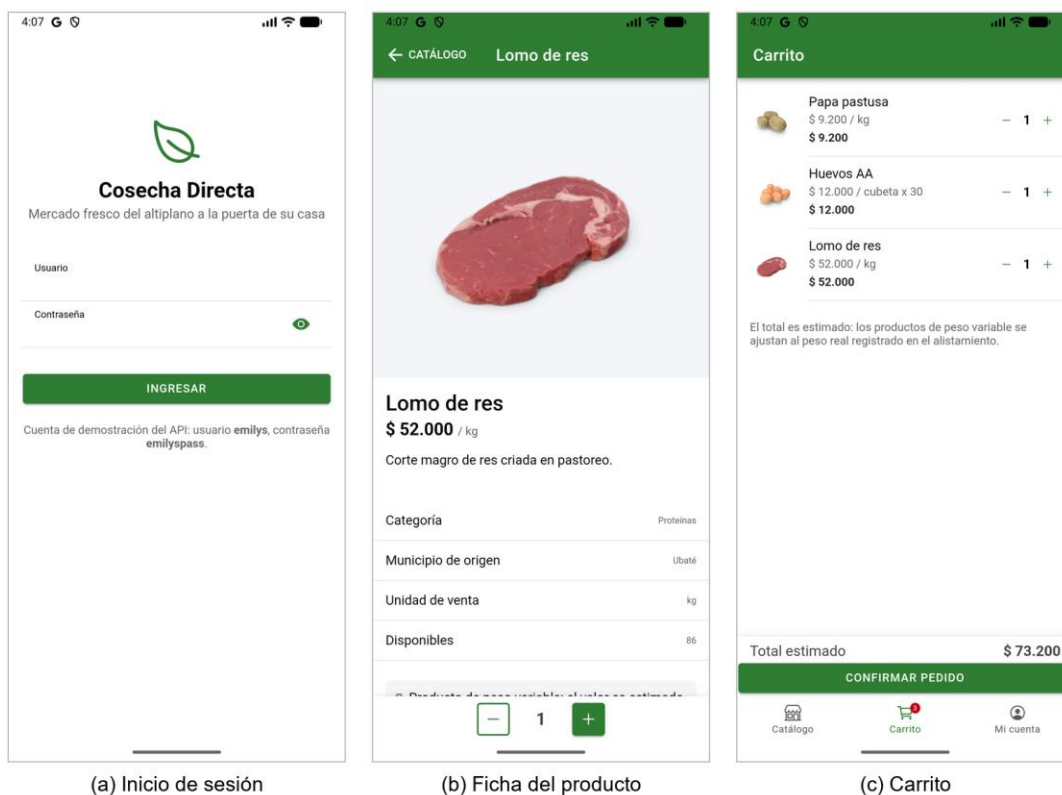
Casos de prueba de extremo a extremo en el emulador

ID	Caso	Resultado esperado	Estado
E-01	Abrir la aplicación con los datos borrados	Pantalla de inicio de sesión	Superado
E-02	Ingresar con la cuenta de demostración	Catálogo con 20 productos desde el API	Superado
E-03	Agregar productos y abrir la ficha del lomo de res	Aviso de producto de peso variable	Superado
E-04	Revisar el carrito	3 productos, total estimado de \$ 73.200	Superado
E-05	Forzar el cierre y reabrir la aplicación	Sesión activa y el mismo carrito con el mismo total	Superado
E-06	Confirmar el pedido	Alerta «Pedido recibido»	Superado
E-07	Revisar «Mi cuenta»	Un pedido en el historial	Superado
E-08	Cerrar sesión	Pantalla de inicio; SharedPreferences sin la clave sesion y con carrito, catalogo y pedidos	Superado

Nota. Emulador Medium Phone con Android 16 (API 36.1). Elaboración propia.

Figura 4

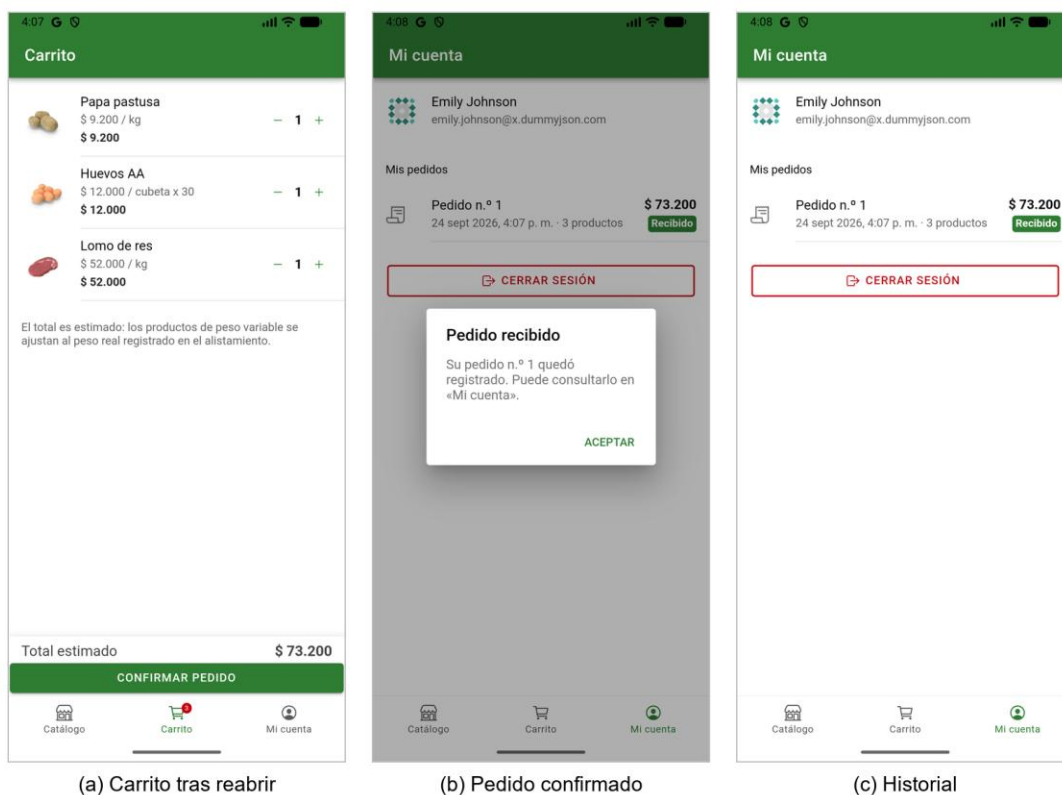
Flujo de compra en el emulador de Android



Nota. Capturas de la pantalla del emulador tomadas con adb screencap. Captura propia.

Figura 5

Persistencia tras el cierre forzado del proceso en Android



Nota. La captura (a) se tomó después de ejecutar am force-stop y volver a abrir la aplicación.

Captura propia.

Prueba en dispositivo físico

La prueba en un teléfono Android físico no se ejecutó en esta entrega porque no se contó con un dispositivo conectado durante la sesión de pruebas. El script del emulador funciona sin cambios con un teléfono conectado por USB con la depuración USB activada, ya que adb trata de la misma manera a un emulador y a un dispositivo. Esta prueba queda registrada como pendiente, y es importante porque un teléfono real de gama media tiene menos memoria y una red menos estable que el emulador.

Prueba de la compilación de producción

La última verificación se hizo sobre el APK de release, firmado con la llave de producción. Se desinstaló la versión de depuración, se instaló la de producción y se revisaron los puntos de la Tabla 7. El tiempo de arranque en frío fue de 1,38 segundos, por debajo de los dos segundos que exige el requerimiento RNF-01 (medido en el emulador, que es más rápido que un teléfono de gama media). La Figura 6 muestra la aplicación de producción en funcionamiento y su ícono en el cajón de aplicaciones.

Tabla 7

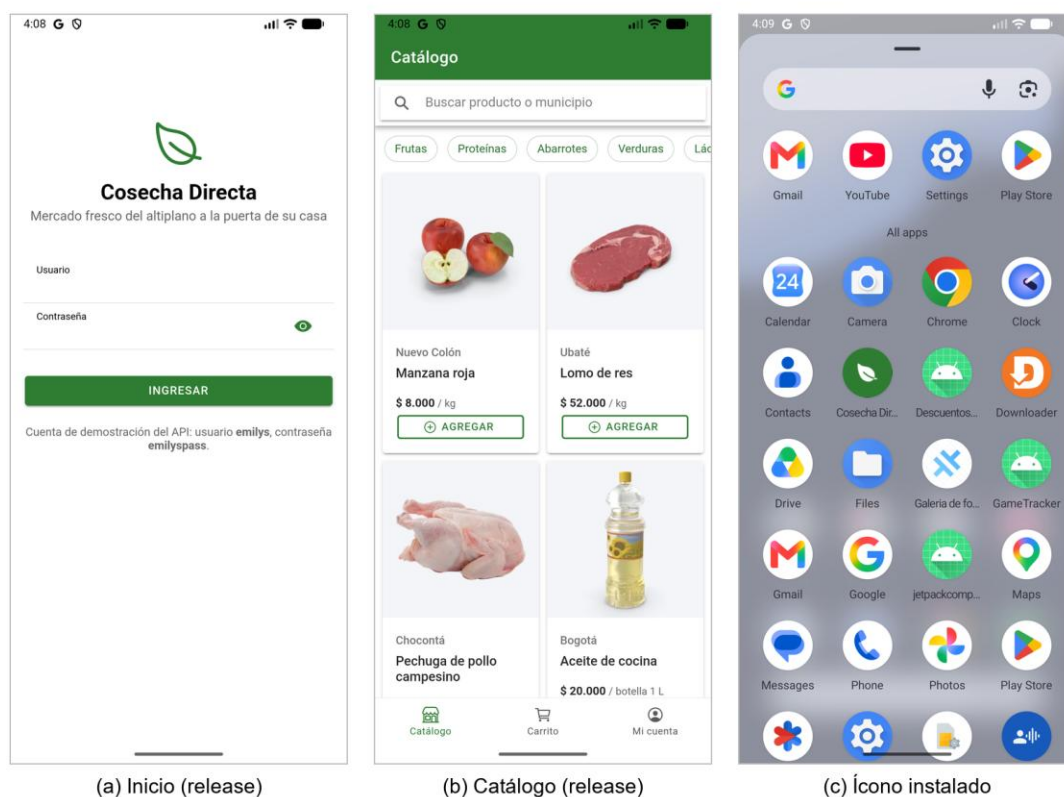
Verificaciones de la compilación de producción

Verificación	Método	Resultado
Firma del APK	apksigner verify	Válida, esquema v2, certificado CN=Samuel Felipe Barbosa Rios, O=Cosecha Directa S.A.S.
Firma del AAB	jarsigner -verify	jar verified, mismo certificado
Identidad del paquete	aapt2 dump badging	co.cosechadirecta.tienda, versionCode 1, versionName 1.0.0, etiqueta «Cosecha Directa»
Instalación y arranque	adb install; am start -W	Estado ok, tiempo total de 1.382 ms
Depuración del WebView	Búsqueda del socket webview_devtools_remote	No existe: el WebView no se puede inspeccionar
Aplicación depurable	adb shell run-as	«package not debuggable»
Estabilidad	logcat -b crash	Cero cierres inesperados; inicio de sesión y catálogo funcionando

Nota. Elaboración propia.

Figura 6

Compilación de producción instalada en el emulador



Nota. APK de release firmado con la llave de producción. Captura propia.

Depuración de errores

Las pruebas encontraron cuatro defectos antes de la entrega (Tabla 8). El más grave fue el del guard de autenticación: en Angular, la función inject() solo se puede llamar antes del primer await, porque después ya no existe contexto de inyección. El guard pedía el enrutador después de esperar a que se restaurara la sesión, de modo que, sin sesión, lanzaba un error en lugar de redirigir al inicio de sesión. La prueba unitaria lo detectó en la primera ejecución, antes de que el defecto llegara al navegador.

Tabla 8*Defectos encontrados y su corrección*

Defecto	Cómo se detectó	Causa	Corrección
El guard fallaba al redirigir a un usuario sin sesión	Prueba unitaria auth.guard.spec.ts	inject(Router) se llamaba después de un await, fuera del contexto de inyección	Obtener el enrutador antes del primer await
Todos los pedidos mostraban el mismo número (209)	Prueba en el emulador, al revisar el historial	El API de demostración devuelve siempre el mismo identificador de carrito	Número consecutivo local y el identificador del API guardado como referencia
Las categorías se mostraban cortadas («Proteína», «Lácteos y hue»)	Revisión de las capturas del navegador	Los chips se encogían dentro del contenedor flexible con desplazamiento horizontal	flex-shrink: 0 en los chips
Prueba intermitente del interceptor con cobertura activa	Ejecución de la suite con --coverage	La prueba esperaba un solo ciclo de eventos, pero la renovación guarda la sesión de forma asíncrona antes de repetir la petición	Esperar activamente a que llegue cada petición, con un límite

Nota. Cada corrección se verificó volviendo a ejecutar la suite completa. Elaboración propia.

En cuanto a la optimización del rendimiento, cada página se carga de forma diferida (lazy loading), por lo que el paquete inicial pesa 208 kB comprimidos y el código de cada pantalla se descarga solo cuando se necesita. Las imágenes del catálogo usan carga diferida del navegador, y el catálogo se guarda localmente para no volver a descargarlo cuando no hay red. El APK de producción pesa 3,8 MB y el AAB, 3,6 MB.

Preparación para el lanzamiento

Antes de generar los archivos de producción se completaron las tareas que las tiendas exigen o revisan: un identificador de aplicación único y definitivo (co.cosechadirecta.tienda), el nombre visible «Cosecha Directa», el número de versión 1.0.0 con código de versión 1, el ícono adaptable de Android, los íconos de iOS y la pantalla de inicio. Los íconos y las pantallas de inicio se generaron para las dos plataformas con la herramienta @capacitor/assets a partir de una sola imagen fuente.

Android: paquete firmado

Google Play exige publicar las aplicaciones nuevas en formato Android App Bundle (AAB) y firmadas (Android Developers, s.f.-b). Se creó una llave de subida (upload key) RSA de 2.048 bits con validez de 10.000 días usando keytool, y se configuró Gradle para leer sus datos desde el archivo keystore.properties. Ni la llave ni ese archivo se incluyen en el repositorio: si la llave se filtra, cualquiera podría firmar una versión falsa de la aplicación. Con el comando `gradlew bundleRelease` se generó el archivo `app-release.aab`, que es el que se sube a Play Console. Este procedimiento equivale al asistente Build > Generate Signed Bundle / APK de Android Studio.

Al publicar, Google Play activa la firma de aplicaciones de Play (Play App Signing): Google guarda la llave de firma definitiva y el desarrollador solo conserva la llave de subida. Si esta última se pierde, se puede solicitar su restablecimiento, cosa que no sería posible si el desarrollador firmara directamente con la llave definitiva (Android Developers, s.f.-b).

iOS: proyecto nativo

Capacitor generó el proyecto de Xcode en la carpeta ios, con la aplicación web ya copiada, el nombre visible, los íconos y las dependencias gestionadas por Swift Package Manager. La compilación para iOS requiere un equipo con macOS y Xcode, el entorno de desarrollo integrado de Apple (Apple, s.f.-c), por lo que no se pudo generar el archivo de iOS desde el equipo con Windows usado en el proyecto. El proyecto queda listo para abrirse en Xcode con el comando `npx cap open ios`.

Proceso de publicación en las tiendas

El enunciado de la actividad excluye la publicación efectiva cuando no se cuenta con las membresías, y este proyecto no las tiene. Esta sección describe el proceso que se seguiría en cada tienda y el estado en que queda cada paso (Tabla 9).

Google Play Console

1. Crear la cuenta de desarrollador como persona o como organización y pagar la cuota única de registro de 25 USD (Google, s.f.).
2. Crear la aplicación en Play Console: nombre, idioma predeterminado, tipo (aplicación) y si es gratuita.
3. Completar la ficha de la tienda (descripción, capturas, ícono de 512 × 512 y gráfico de funciones), la clasificación de contenido, la sección de seguridad de los datos y la URL de la política de privacidad, que en Colombia debe cumplir la Ley 1581 de 2012.
4. En Pruebas o Producción, crear una versión y subir el archivo app-release.aab.
5. Enviar la versión a revisión. Según la política vigente, las cuentas personales nuevas deben hacer primero una prueba cerrada con al menos 12 evaluadores durante 14 días antes de solicitar el acceso a producción (Google, s.f.).

App Store Connect

1. Inscribirse en el Apple Developer Program, que cuesta 99 USD al año (Apple, s.f.-b).
2. Registrar el identificador co.cosechadirecta.tienda y crear la aplicación en App Store Connect con su nombre, idioma, SKU y categoría (Apple, s.f.-a).
3. En Xcode, seleccionar el equipo de firma, compilar con Product > Archive y, desde el Organizer, elegir Distribute App > App Store Connect.

- 4. Probar la compilación con TestFlight, completar la información de la aplicación (capturas, descripción, privacidad) y enviarla a revisión de Apple.

Tabla 9

Estado de los pasos de publicación

Paso	Google Play	App Store
Identificador, nombre, versión e íconos	Hecho	Hecho
Proyecto nativo generado	Hecho (android/)	Hecho (ios/)
Llave de firma	Hecho (llave de subida)	Requiere membresía (certificado de distribución)
Archivo de producción	Hecho (app-release.aab firmado)	Requiere macOS y Xcode (.ipa)
Cuenta de desarrollador	No disponible (25 USD, pago único)	No disponible (99 USD al año)
Ficha, privacidad y envío a revisión	Pendiente de la cuenta	Pendiente de la cuenta

Nota. Elaboración propia.

Consideraciones de seguridad

Las principales amenazas de una aplicación móvil incluyen el almacenamiento inseguro de datos, la comunicación insegura y una autenticación deficiente (OWASP Foundation, 2024).

La aplicación aplica las siguientes medidas:

- **Comunicación cifrada.** Todas las peticiones al API usan HTTPS, y Android bloquea por defecto el tráfico sin cifrar desde la versión 9.
- **Token acotado.** El token se adjunta solo a las peticiones dirigidas al API; el interceptor no lo envía a otros dominios, como el de las imágenes. Así lo verifica una prueba unitaria.
- **Sesión con vencimiento.** El token de acceso vence a los 30 minutos (RNF-07) y se renueva una sola vez con el token de actualización; si la renovación falla, se cierra la sesión.

- **Compilación de producción cerrada.** El APK de release no permite inspeccionar el WebView ni ejecutar comandos con run-as, como se comprobó en la prueba de humo.
- **Llave de firma fuera del repositorio,** excluida mediante .gitignore.
- **Sin datos de pago en el dispositivo.** La aplicación no procesa tarjetas, lo que es coherente con el requerimiento RNF-06.

Queda una limitación conocida: Preferences guarda los datos sin cifrar. Para una versión comercial, la sesión debería guardarse en el almacenamiento seguro del sistema (Keystore en Android y Keychain en iOS) mediante un complemento de almacenamiento seguro, y la aplicación debería usar un servidor propio en lugar del API de demostración.

Actualizaciones y mantenimiento

Cada actualización debe incrementar el código de versión (versionCode en Android y número de compilación en iOS), porque las tiendas rechazan un paquete con un número igual o menor al publicado. También debe firmarse con la misma llave de subida, de modo que la llave y sus contraseñas tienen que guardarse con copia de seguridad fuera del equipo de desarrollo. El ciclo de publicación de un cambio consiste en compilar la aplicación web, sincronizarla con los proyectos nativos mediante npx cap sync y generar los paquetes. Antes de cada publicación se ejecuta la suite de pruebas completa, que funciona como prueba de regresión.

Como la aplicación tiene una sola base de código, una corrección se publica al mismo tiempo en las dos tiendas, lo que cumple el requerimiento de portabilidad RNF-15. Las dependencias (Angular, Ionic y Capacitor) deben actualizarse de forma periódica, porque Google Play exige que las aplicaciones apunten a una versión reciente de Android para seguir recibiendo actualizaciones.

Conclusiones

La aplicación cumple los cuatro componentes que exige la actividad: autenticación, servicios API, carrito de compras y almacenamiento local. Las 33 pruebas automatizadas y los dos recorridos de extremo a extremo se superaron en el navegador y en Android. La cobertura de la lógica de negocio (94,5 %) supera la meta del 70 % planteada en los requerimientos.

La principal lección es que cada nivel de prueba encontró defectos que los otros no habrían detectado. La prueba unitaria encontró el fallo del guard sin necesidad de abrir la aplicación. La prueba en el emulador mostró que el API repetía el número de pedido, algo que en el navegador pasaba desapercibido con un solo pedido. Y la prueba de humo confirmó que el paquete de producción, distinto al de desarrollo, arranca, está bien firmado y no expone la depuración.

El archivo de producción de Android queda listo para subirse a Google Play Console. El de iOS depende de un equipo con macOS y de la membresía de Apple. Quedan pendientes la prueba en un dispositivo físico, el almacenamiento cifrado de la sesión y la publicación efectiva, que requiere las membresías de desarrollador.

Referencias

Android Developers. (s.f.-a). *Android Studio*. Recuperado el 24 de septiembre de 2026, de <https://developer.android.com/studio>

Android Developers. (s.f.-b). *Firmar tu app*. Recuperado el 24 de septiembre de 2026, de <https://developer.android.com/studio/publish/app-signing>

Angular. (s.f.). *Testing*. Recuperado el 24 de septiembre de 2026, de <https://angular.dev/guide/testing>

Apple. (s.f.-a). *App Store Connect*. Recuperado el 24 de septiembre de 2026, de <https://developer.apple.com/app-store-connect/>

Apple. (s.f.-b). *Apple Developer Program*. Recuperado el 24 de septiembre de 2026, de <https://developer.apple.com/programs/>

Apple. (s.f.-c). *Xcode* [Aplicación]. Mac App Store. Recuperado el 24 de septiembre de 2026, de <https://apps.apple.com/co/app/xcode/id497799835?mt=12>

Barbosa Ríos, S. F. (2026). *Actividad 2: caso Cosecha Directa S.A.S.* [Trabajo académico no publicado]. Fundación Universitaria Compensar.

Capacitor. (s.f.-a). *Capacitor: Cross-platform native runtime for web apps*. Recuperado el 24 de septiembre de 2026, de <https://capacitorjs.com/docs>

Capacitor. (s.f.-b). *Preferences Capacitor plugin API*. Recuperado el 24 de septiembre de 2026, de <https://capacitorjs.com/docs/apis/preferences>

Cohn, M. (2009). *Succeeding with agile: Software development using Scrum*. Addison-Wesley.

DummyJSON. (s.f.). *DummyJSON documentation*. Recuperado el 24 de septiembre de 2026, de <https://dummyjson.com/docs>

Google. (s.f.). *Ayuda de Play Console*. Recuperado el 24 de septiembre de 2026, de

<https://support.google.com/googleplay/android-developer>

Ionic. (s.f.). *Introduction to Ionic*. Ionic Framework Docs. Recuperado el 24 de septiembre de

2026, de <https://ionicframework.com/docs>

Microsoft. (s.f.). *Playwright for Python*. Recuperado el 24 de septiembre de 2026, de

<https://playwright.dev/python/>

OWASP Foundation. (2024). *OWASP Mobile Top 10*. [https://owasp.org/www-project-mobile-](https://owasp.org/www-project-mobile-top-10/)

[top-10/](https://owasp.org/www-project-mobile-top-10/)

Vitest. (s.f.). *Vitest: Next generation testing framework*. Recuperado el 24 de septiembre de

2026, de <https://vitest.dev>